



GABRYELLA SANTOS PINHO
LOUIE NERY SILVA
MARIA EDUARDA RITA MARQUES NOLETO
NATHANAEL VICTOR PAIVA MAGNO
RAMON MIGUEL ROSA PEREIRA ATAIDES

APLICAÇÃO DE TÉCNICAS DE MACHINE LEARNING NA CLASSIFICAÇÃO DE GÊNEROS MUSICAIS EM PLATAFORMAS DE STREAMING

Artigo apresentado ao curso de graduação em Análise e Desenvolvimento de Sistemas da Universidade Católica de Brasília, como requisito parcial para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Orientador: William Roberto Malvezzi

Brasília

2026

Sumário

Introdução.....	3
Fundamentação Teórica.....	4
Recuperação de Informação Musical (MIR).....	4
Algoritmos de Classificação.....	4
R e Python na Ciência de Dados.....	5
Metodologia.....	6
Descrição da Base de Dados.....	6
Etapas de Pré-processamento.....	8
Estratégia Experimental.....	8
Implementação.....	9
Desenvolvimento em Python.....	9
Desenvolvimento em R.....	9
Resultados.....	11
Resultados em Python.....	11
Resultados em R.....	12
Comparação Entre as Duas Linguagens.....	13
Discussão.....	14
Conclusão.....	15
Referências.....	16

Introdução

O crescimento acelerado das plataformas de streaming musical trouxe consigo o desafio de organizar e categorizar automaticamente um grande volume de novas faixas. A classificação manual de músicas em gêneros torna-se impraticável diante dessa escala, exigindo soluções automatizadas baseadas em aprendizado de máquina. Nesse contexto, modelos computacionais são capazes de identificar padrões e características relevantes nas músicas, permitindo uma categorização eficiente a partir de dados previamente rotulados.

A correta classificação de gêneros musicais desempenha um papel fundamental na melhoria dos sistemas de recomendação, amplamente utilizados por essas plataformas. Modelos mais precisos possibilitam sugestões mais alinhadas ao perfil do usuário, aumentando o engajamento, a satisfação e a descoberta de novos conteúdos. Além disso, a organização adequada do catálogo musical contribui para uma navegação mais intuitiva e eficiente dentro das aplicações.

Diante disso, este trabalho tem como objetivo desenvolver e avaliar modelos de aprendizado de máquina para a classificação automática de gêneros musicais, utilizando algoritmos como Naive Bayes, Árvores de Decisão e Random Forest. A metodologia envolve o uso de uma base de dados rotulada, etapas de pré-processamento, definição de critérios de avaliação e implementação nas linguagens Python e R. Por fim, os resultados obtidos são analisados por meio de métricas e representações gráficas, permitindo uma discussão crítica sobre o desempenho dos modelos e a identificação da abordagem mais adequada para o problema proposto.

Fundamentação Teórica

Recuperação de Informação Musical (MIR)

A base deste trabalho fundamenta-se na Recuperação de Informação Musical (*Music Information Retrieval* - *MIR*), uma área interdisciplinar que utiliza o processamento de sinais digitais para extrair informações significativas de arquivos de áudio. No contexto da API do Spotify, as “Audio Features” representam a tradução de ondas sonoras em atributos numéricos. Características como *Acousticness* ou *Instrumentalness* são resultados de modelos de MIR que analisam o timbre e a harmonia, permitindo que algoritmos de Machine Learning identifiquem padrões de gênero musical sem a necessidade de intervenção humana direta.

Algoritmos de Classificação

- Naive Bayes: Baseia-se no **Teorema de Bayes** com a premissa de independência entre as variáveis (como *Energy* e *Acousticness*). Por trabalhar com probabilidades em vez de regras rígidas, é ideal para lidar com a subjetividade e a sobreposição de gêneros musicais, indicando a chance de uma faixa pertencer a cada categoria.
- Árvore de Decisão: Estrutura o aprendizado como um fluxograma de decisões lógicas. O algoritmo realiza perguntas sucessivas sobre os atributos (ex: "A energia é > 0.8 ?") para classificar os dados. Sua principal vantagem é a *interpretabilidade*, permitindo visualizar claramente os critérios técnicos que levaram à escolha de um gênero.
- Random Forest: É um método de aprendizado por conjunto (*ensemble*) que constrói múltiplas Árvores de Decisão de forma aleatória. O resultado final é definido pela "votação" da maioria das árvores, o que torna o modelo muito mais robusto e assertivo, reduzindo erros de generalização comuns em árvores únicas.

Todos os algoritmos tem o retorno de:

- Precision: mede a confiabilidade das previsões.
- Accuracy: avalia o desempenho global.
- Recall: indica a capacidade de detectar corretamente as classes.
- F1-score: oferece uma visão equilibrada e robusta do modelo.

R e Python na Ciência de Dados

No contexto deste projeto de classificação de gêneros musicais, tanto R quanto Python se mostram ferramentas eficazes para Ciência de Dados, porém com características distintas. O R é tradicionalmente mais voltado para análise estatística e exploração de dados, oferecendo grande facilidade na visualização e manipulação por meio de pacotes como *ggplot2* e *dplyr*. Isso o torna especialmente útil nas etapas iniciais do projeto, como análise exploratória e interpretação de padrões presentes nas features musicais.

Por outro lado, o Python apresenta maior versatilidade e integração com aplicações reais, sendo amplamente utilizado em projetos de aprendizado de máquina. Bibliotecas como *scikit-learn*, *pandas* e *NumPy* permitem implementar modelos como Naive Bayes, Árvores de Decisão e Random Forest de forma eficiente e escalável. Além disso, o Python facilita a integração com sistemas de produção, o que é essencial em cenários como plataformas de streaming.

De forma geral, enquanto o R se destaca na análise estatística e visualização, o Python é mais adequado para implementação, escalabilidade e integração de modelos. Assim, neste projeto, o uso combinado de ambas as linguagens pode ser estratégico: R para análise e compreensão dos dados, e Python para construção e aplicação dos modelos de classificação.

Metodologia

A metodologia deste trabalho consiste em uma abordagem quantitativa e comparativa entre as linguagens Python e R, aplicando modelos de aprendizado de máquina sobre dados de áudio.

Descrição da Base de Dados

A base de dados utilizada foi o *Spotify Music Dataset*, obtida originalmente via Kaggle (conforme detalhado na seção de “Referências”). O dataset contém metadados e características acústicas extraídas pela API oficial do Spotify. As principais variáveis utilizadas na análise foram:

- **Speechiness** - A fala indica o nível de presença de palavras faladas em uma faixa. Quanto mais a gravação se assemelha exclusivamente à fala (como talk shows, audiolivros ou poesia), mais próximo de 1,0 será esse valor. Valores acima de 0,66 indicam faixas predominantemente faladas; entre 0,33 e 0,66 representam conteúdos mistos, com música e fala (como no rap); e abaixo de 0,33 correspondem, em geral, a músicas com pouca ou nenhuma presença de fala.
- **Acousticness** - uma medida de confiança de 0,0 a 1,0 sobre se a faixa é acústica. A versão 1.0 representa alta confiança de que a faixa é acústica.
- **Valence** - um compasso de 0,0 a 1,0 descrevendo a positividade musical transmitida por uma faixa. Faixas com alta valência soam mais positivas (por exemplo, felizes, alegres, eufóricas), enquanto faixas com baixa valência soam mais negativas (por exemplo, tristes, deprimidas, irritadas).
- **Danceability** - descreve o quão adequada é uma faixa para dançar com base em uma combinação de elementos musicais, incluindo tempo, estabilidade rítmica, força do ritmo e regularidade geral. Um valor de 0,0 é o menos dançável e 1,0 é o mais dançável.

- Energy - é uma medida de 0,0 a 1,0 e representa uma medida perceptiva de intensidade e atividade. Normalmente, faixas energéticas soam rápidas, altas e barulhentas. Por exemplo, death metal tem alta energia, enquanto um prelúdio de Bach tem pontuação baixa na escala. Características perceptuais que contribuem para esse atributo incluem alcance dinâmico, volume percebido, timbre, taxa de início e entropia geral.
- Time - o tempo estimado geral de uma faixa em batimentos por minuto (BPM). Na terminologia musical, tempo é a velocidade ou ritmo de uma determinada peça e deriva diretamente da duração média do tempo.
- Playlist_genre - É uma variável categórica que classifica a playlist na qual a música está inserida em gêneros principais. No dataset do Spotify, as categorias comuns incluem Pop, Rock, Latin, R&B e EDM. Essa variável é essencial para análises comparativas entre nichos musicais e comportamentos de consumo.
- Instrumentalness - Representa a probabilidade de uma faixa não conter vocais. Sons como “ooh” e “aah” são tratados como instrumentais neste contexto, enquanto faixas de rap ou poesia falada são claramente "vocais". Quanto mais próximo de 1,0, maior a confiança de que a música seja puramente instrumental. Valores acima de 0,5 geralmente indicam faixas sem voz, mas a confiança aumenta à medida que o valor se aproxima do topo da escala.
- Duration_minutes - Indica o tempo total de duração da faixa convertido para minutos. Embora a API do Spotify forneça originalmente esse dado em milissegundos (*duration_ms*), a conversão para minutos é frequentemente realizada para facilitar a leitura e a interpretação estatística sobre a extensão média das composições.

Etapas de Pré-processamento

Para garantir a consistência entre os experimentos em ambas as linguagens, os dados passaram pelas seguintes etapas:

- Limpeza: Remoção de registros duplicados e tratamento de valores.
- Seleção de Atributos: Exclusão de variáveis identificadoras (como `track_id` e `artist_name`) que não contribuem para a generalização do modelo.
- Normalização: Aplicação de escala para converter variáveis de diferentes magnitudes para um intervalo comum, evitando que atributos como Tempo (0-200+) dominem atributos como Valence (0-1).

Estratégia Experimental

Para garantir a validade da comparação entre as linguagens Python e R, foi adotado um protocolo experimental rigoroso, assegurando que os modelos fossem treinados e testados sob as mesmas condições:

- Divisão da Base de Dados: O dataset foi particionado utilizando a proporção de 70% para o conjunto de treinamento (destinado ao aprendizado dos padrões) e 30% para o conjunto de teste (destinado à validação e medição de acurácia).
- Reprodutibilidade (Semente Aleatória): Para assegurar que ambas as linguagens selecionassem exatamente as mesmas amostras de músicas para treino e teste, foi fixada uma semente aleatória (seed) idêntica em ambos os ambientes. Isso elimina variações estatísticas causadas pelo acaso, permitindo uma comparação direta entre os algoritmos.

Implementação

Nesta etapa, o ambiente de desenvolvimento e as ferramentas específicas utilizadas para a construção dos modelos de classificação. O objetivo foi replicar a lógica experimental em ambos os ecossistemas (Python e R) para validar a consistência dos resultados.

Desenvolvimento em Python

A implementação em Python focou na modularidade e eficiência do processo de treinamento. Foram utilizadas as seguintes bibliotecas:

- **Pandas:** Utilizada para a manipulação e tratamento dos dados, incluindo limpeza do dataset, criação de variáveis derivadas e preparação das bases de treino e teste.
- **Scikit-learn (Sklearn):** Empregada na construção da pipeline de Machine Learning, abrangendo pré-processamento com imputação de valores ausentes e codificação de variáveis categóricas, além da implementação do algoritmo Random Forest, divisão dos dados e cálculo das métricas de avaliação.
- **Joblib:** Utilizada para a serialização e persistência do modelo treinado e da pipeline, permitindo o armazenamento e a reutilização eficiente dos artefatos gerados.

Desenvolvimento em R

A implementação em R foi desenvolvida com foco na reprodutibilidade e na comparação direta com os resultados obtidos em Python. Foram utilizados os seguintes pacotes:

- Tidyverse (dplyr, readr, ggplot2, lubridate): Utilizado como a base para todo o ciclo de ciência de dados. O readr foi empregado na leitura rápida dos CSVs; o dplyr na limpeza, filtragem e criação de variáveis derivadas; o lubridate no tratamento de datas; e o ggplot2 na geração dos gráficos comparativos de métricas.
- Caret (Classification and Regression Training): Esta é a biblioteca central de avaliação. Foi utilizada para a criação da partição de dados (divisão

treino/teste) de forma estratificada e para a geração da Confusion Matrix, que fornece automaticamente a Acurácia, Precisão, Recall e F1-Score.

- naivebayes: Diferente do pacote básico e1071, este pacote foi escolhido por ser mais eficiente para lidar com grandes datasets e por permitir o uso de Kernel Density Estimation (KDE), o que melhora a precisão em dados musicais não lineares.
- rpart (Recursive Partitioning): Biblioteca padrão para a implementação da Árvore de Decisão via algoritmo CART. Permite o controle rigoroso da poda da árvore através dos parâmetros maxdepth (profundidade) e cp (complexidade), evitando o *overfitting*.

Resultados

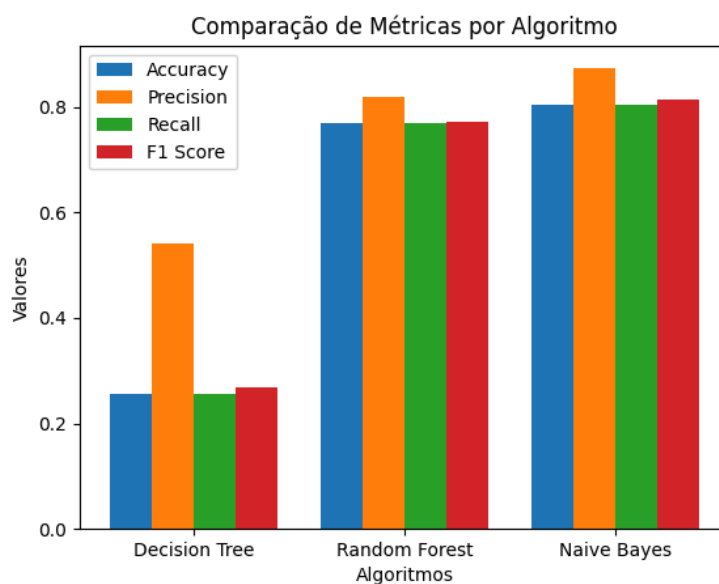
Resultados em Python

A tabela a seguir apresenta a avaliação comparativa de desempenho dos algoritmos Decision Tree, Random Forest e Naive Bayes, implementados em Python, com base nas métricas de acurácia (*accuracy*), precisão (*precision*), revocação (*recall*) e medida F1 (*f1-score*).

Algoritmos	accuracy	Precision	Recall	f1_score
Decision Tree	0.25695 (25%)	0.54190 (54%)	0.25695 (25%)	0.26851 (26%)
Random Forest	0.76974 (76%)	0.81916 (81%)	0.76974 (76%)	0.77206 (77%)
Naive Bayes	0.80533 (80%)	0.87342 (87%)	0.80533 (80%)	0.81473 (81%)

* Os valores da tabela estão sendo representados com 5 casas decimais e suas porcentagens.

O gráfico a seguir ilustra a comparação do desempenho dos algoritmos Decision Tree, Random Forest e Naive Bayes, implementados em Python, com base nas métricas e critérios de avaliação acima, permitindo uma análise visual da performance relativa entre os modelos.



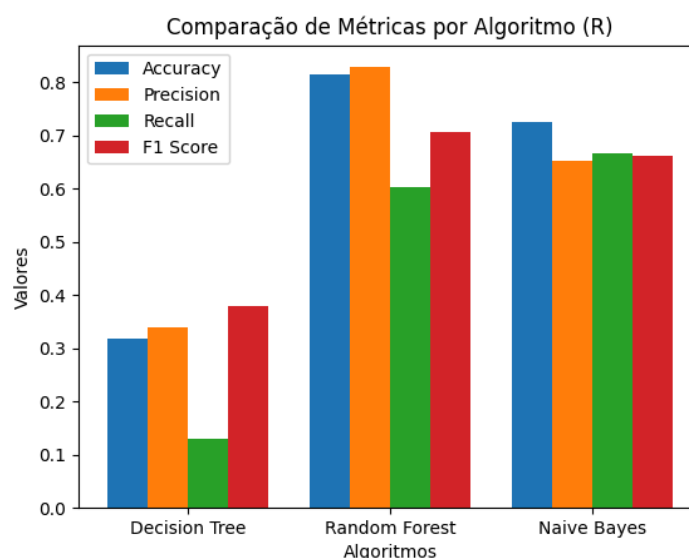
Resultados em R

Em seguida, apresenta-se a avaliação dos mesmos algoritmos implementados na linguagem R, mantendo-se os mesmos parâmetros experimentais e métricas de desempenho (acurácia, precisão, revocação e F1-score), com o objetivo de possibilitar uma análise comparativa consistente entre os ambientes de desenvolvimento.

Algoritmos	accuracy	Precision	Recall	f1_score
Decision Tree	0.31707 (31%)	0.33840 (33%)	0.13028 (13%)	0.38043 (38%)
Random Forest	0.81489 (81%)	0.82827 (82%)	0.60316 (60%)	0.70545 (70%)
Naive Bayes	0.72590 (72%)	0.65275 (65%)	0.66636 (66%)	0.66213 (66%)

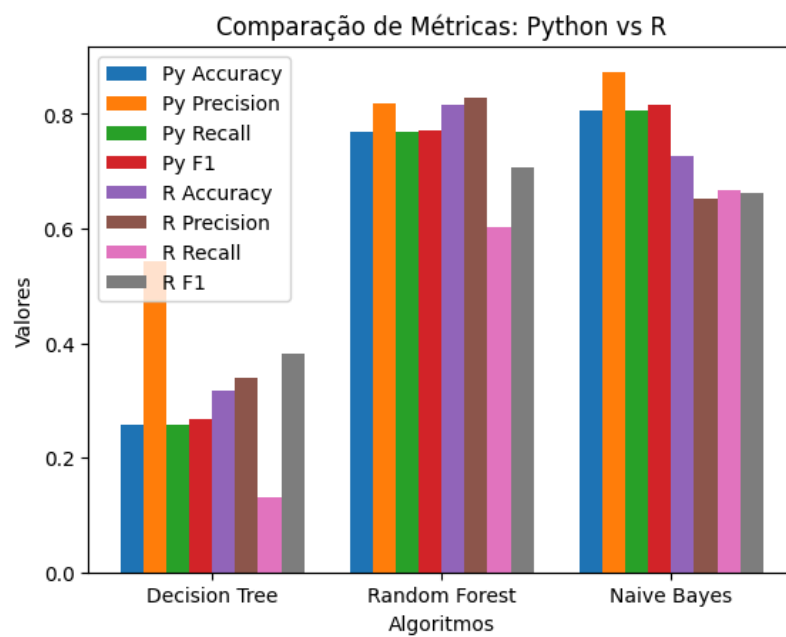
* Os valores da tabela estão sendo representados com 5 casas decimais e suas porcentagens.

O gráfico a seguir apresenta a comparação do desempenho dos algoritmos Decision Tree, Random Forest e Naive Bayes na linguagem R, considerando as métricas e critérios de avaliação previamente descritos, permitindo a visualização do comportamento dos modelos neste ambiente.



Comparação Entre as Duas Linguagens

Por fim, segue a comparação das métricas entre as duas linguagens a fim de permitir identificar variações decorrentes do ambiente de execução e das bibliotecas utilizadas.



Discussão

A análise comparativa dos resultados evidencia diferenças relevantes tanto entre os algoritmos quanto entre os ambientes de execução (Python e R), permitindo uma avaliação mais crítica do desempenho dos modelos.

De forma geral, o Naive Bayes apresentou o melhor desempenho no ambiente Python, com valores elevados e consistentes em todas as métricas (accuracy $\approx$ 80%, precision $\approx$ 87%, recall $\approx$ 80% e F1-score $\approx$ 81%). Esse comportamento indica um modelo bem balanceado, com boa capacidade tanto de identificar corretamente as classes quanto de minimizar falsos positivos. No entanto, ao observar os resultados em R, há uma queda em todas as métricas, especialmente na precisão e no F1-score, sugerindo sensibilidade à implementação, pré-processamento ou diferenças nos parâmetros padrão entre as bibliotecas utilizadas.

O Random Forest demonstrou alta robustez em ambos os ambientes, mantendo métricas elevadas e relativamente estáveis. Em R, inclusive, apresentou a maior acurácia ($\approx$ 81%) e precisão ($\approx$ 82%), o que reforça sua capacidade de generalização. Entretanto, observa-se uma redução significativa no recall ($\approx$ 60%), indicando que, embora o modelo seja preciso, ele pode estar deixando de identificar uma parcela relevante dos casos positivos (maior incidência de falsos negativos). Em Python, o equilíbrio entre precision e recall é mais evidente, resultando em um F1-score consistente ($\approx$ 77%).

Já o Decision Tree apresentou o pior desempenho geral em ambos os cenários. Em Python, apesar de uma precisão relativamente moderada ($\approx$ 54%), a baixa acurácia e recall ($\approx$ 25%) indicam um modelo com baixa capacidade de generalização e alta taxa de erro. Em R, embora a acurácia tenha melhorado ligeiramente ($\approx$ 31%), o recall caiu drasticamente ($\approx$ 13%), evidenciando um modelo ainda mais incapaz de identificar corretamente os casos positivos. Esse comportamento sugere possível overfitting ou inadequação da árvore de decisão para a estrutura dos dados analisados.

Comparando diretamente os ambientes, nota-se que o Python apresentou resultados mais consistentes e equilibrados, especialmente para o Naive Bayes e Random Forest. Já o R apresentou maior variabilidade entre as métricas, com destaque para quedas no recall em alguns modelos, o que impacta negativamente o F1-score e, conseqüentemente, a qualidade global da classificação.

Em síntese, os resultados indicam que:

- O Naive Bayes (Python) é a melhor opção em termos de equilíbrio geral.
- O Random Forest é o modelo mais robusto entre ambientes, embora exija atenção ao recall em R.
- O Decision Tree não se mostrou adequado para o problema, independentemente da linguagem.

Essa análise reforça a importância não apenas da escolha do algoritmo, mas também do ambiente, das bibliotecas e das configurações utilizadas, uma vez que tais fatores impactam diretamente o desempenho final dos modelos de aprendizado de máquina.

Conclusão

A classificação automática de gêneros musicais revela-se um desafio complexo devido à natureza subjetiva da música e à sobreposição de características acústicas entre diferentes estilos. Este trabalho cumpriu seu objetivo ao implementar e comparar os algoritmos Naive Bayes, Árvore de Decisão e Random Forest nas linguagens Python e R, utilizando uma base de dados real proveniente da API do Spotify.

Os resultados obtidos demonstram que o algoritmo Random Forest apresentou o desempenho mais robusto e equilibrado em ambos os ambientes de desenvolvimento. Com uma acurácia superior a 80% na implementação em R e métricas sólidas em Python, o modelo de random forest provou ser eficaz ao lidar com a alta dimensionalidade dos dados e com variáveis categóricas complexas (subgêneros), superando as limitações de interpretabilidade da Árvore de Decisão e as premissas de independência do Naive Bayes.

Em suma, conclui-se que o sucesso na classificação de gêneros musicais não depende apenas da escolha do algoritmo, mas de um pré-processamento rigoroso que elimine duplicatas e trate valores ausentes sem corromper a integridade sonora dos dados. Por fim, conclui-se que o sucesso na classificação não depende apenas do algoritmo, mas de um pré-processamento rigoroso que elimine ruídos e duplicatas. Para trabalhos futuros, recomenda-se a exploração de técnicas de ajuste de e inclusão de variáveis contextuais, como a popularidade do artista e o ano de lançamento, para refinar ainda mais a distinção entre gêneros que compartilham atributos sonoros semelhantes.

Referências

- AMEH, Solomon. **Spotify Music Dataset**. [S. l.]: Kaggle, 2022. Disponível em: <https://www.kaggle.com/datasets/solomonameh/spotify-music-dataset/>. Acesso em: 04 abr. 2026.
- SPOTIFY. **Web API Reference**: Spotify for Developers. [S. l.], 2024. Disponível em: <https://developer.spotify.com/documentation/web-api/reference/get-audio-features>. Acesso em: 04 abr. 2026.
- TJOA, Steve. **Introduction to Music Information Retrieval**. [S. l.], [20--]. Disponível em: <https://musicinformationretrieval.com/intro.html>. Acesso em: 04 abr. 2026.